

Gerald Weber

Transaction Processing

Motivation: isolation concepts for concurrency

Example: Flight booking: Concurrency

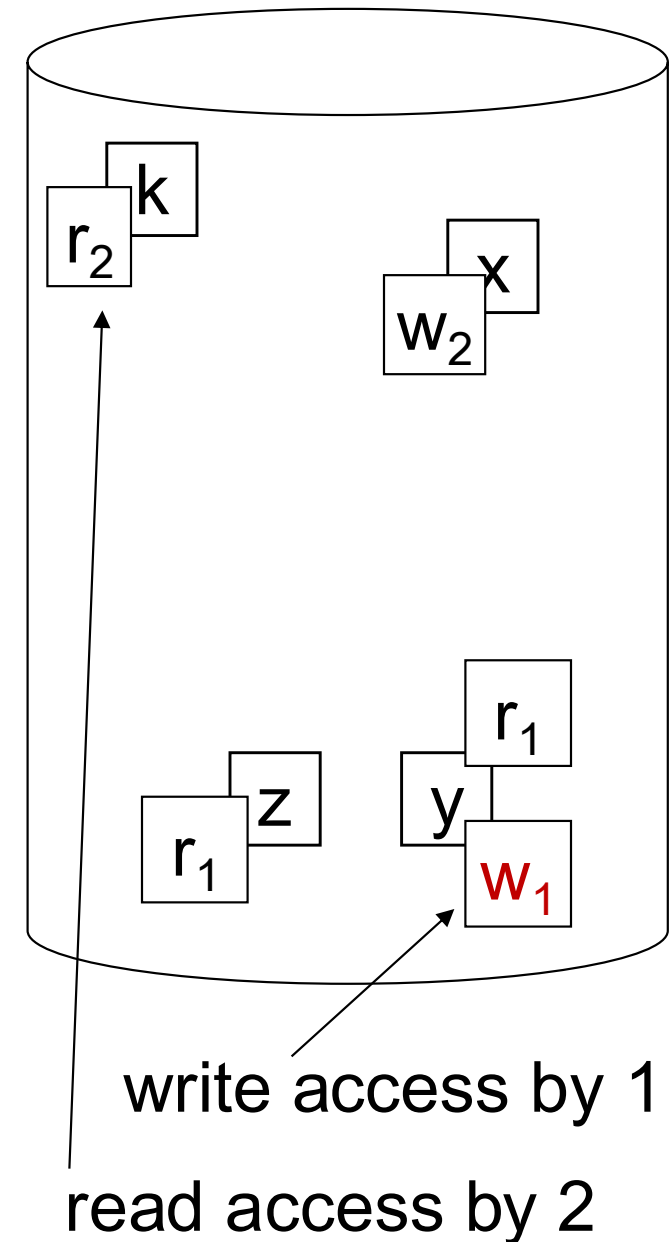
- An online multi-leg flight booking system (web or app).
- Users submit their orders and expect them to be:
 - Processed as quickly as possible, also meaning that the user gets quick feedback on success/no success.
 - Exactly as requested, in particular complete:
 - Book all legs of the flight, or do nothing.
- Problem **Concurrency**: different clients might interfere
 - System checked that all legs were available
 - but while booking: first flight ok, last flight is full.

Disjoint-access parallelism (DAP) [e.g. Israeli et al 1994]

- Shared object: a data object accessed by more than one transaction in a set.
- A set of transactions is disjoint-access, if it has no shared object.
- Disjoint-access transactions should not delay each other, ideal:

Disjoint-access parallelism (DAP)

A. Israeli and L. Rappoport. Disjoint-access-parallel Implementations of Strong Shared Memory Primitives. In PODC, pages 151–160, 1994.



First Solution: simple scheduler [e.g. Ullmann 1980]

- Idea:
- The *simple scheduler* uses Mutex per data object, aka one exclusive lock per object.
- When to set lock?
 - Scheduler knows only at the first access of x by TA1;
- So TA1 gets excl. lock on x with its first operation on x:
 - latest time allowed, first time possible (implicit OLTP).
- Release all locks held by one transaction at end of transaction (commit/abort time)
- If transactions wait in a cycle for each other to release locks: deadlock, abort one transaction in the cycle.

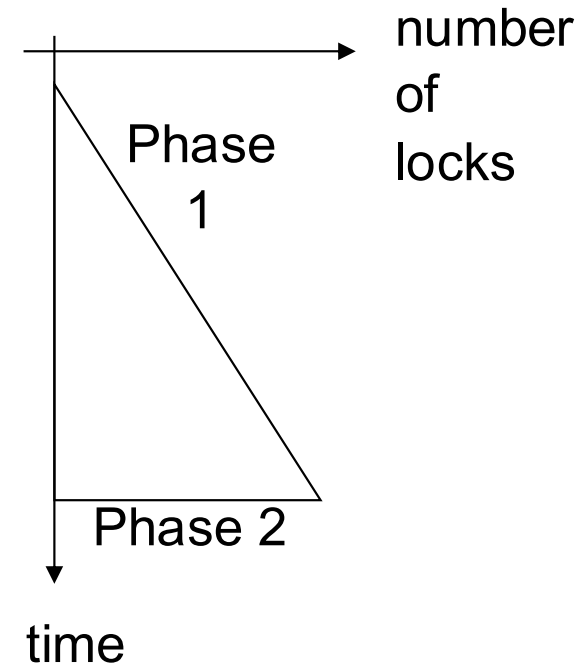
Transactions wait for locks

For a scheduler:

- If a transaction TA1 makes any access to a *locked* object x and gets *blocked*:
- The scheduler does not answer to TA1 for now:
- From the perspective of TA1, the call to access x simply takes long
 - TA1 does not have to care about scheduling
 - best of all possible worlds, solve problem by doing nothing 😊
- Transaction scripts are typically strictly sequential: TA1, while blocked, cannot execute any other operation.

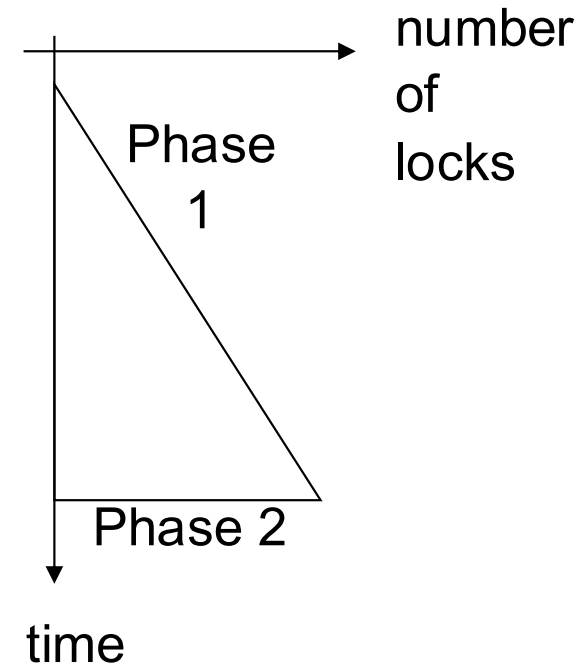
Rigorous two-phase locking (R2PL)

- The simple scheduler as described uses:
- *Rigorous two-phase locking (R2PL):*
- transactions acquire locks during their lifetime (first phase).
- When should one transaction release any lock?
- We do not want to require further info from clients (implicit TP);
- we have only agreed on transaction demarcation:
- So release all locks at end of transaction.



Rigorous two-phase locking (R2PL)

- The simple scheduler as described uses:
- *Rigorous two-phase locking (R2PL):*
- transactions acquire locks during their lifetime (first phase).
- They release all locks immediately after commit/abort, but not earlier (second phase).
- R2PL used by most lock-based schedulers:
- No explicit lock commands necessary (only commit): no break of abstraction for the programmers writing transactions.
- R2PL avoids cascading aborts in case of errors.



The current basic transaction model

- allows precise reasoning on transaction scheduling.
- elementary operations in transaction s :
 - *read* on a data object: $r_s[x]$ - client gets the content
 - *write* on a data object: $w_s[x]$ - client provides new content
- transaction demarcation:..
 - Begin of Transaction (BOT_s) -
 - mostly implicit: first operation starts transaction
 - Commit: c_s - successful end of transaction
 - Abort: a_s - unsuccessful end of transaction

Linear schedules

- A (linear) schedule orders operations of a **set** of transactions in time. *Good for discussing isolation issues.*
(serializability)
- Example:

- Schedule

$s : r_1[x], r_2[x], r_1[y], w_1[x], r_3[y], c_3, w_2[x], a_2, c_1$

- Transaction

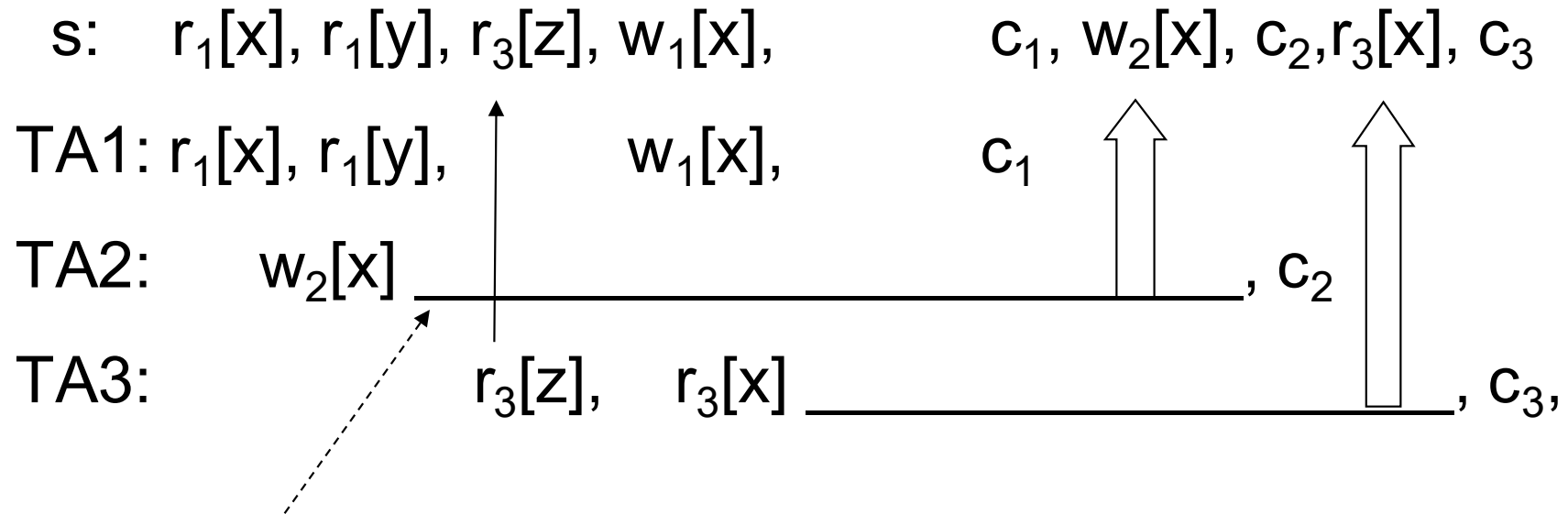
TA1: $r_1[x], r_1[y], w_1[x], c_1$ (*)

- Lines do not cross: the schedule respects the local order in the transaction. (*) is called the local schedule of TA1.

- A transaction is a schedule.

Schedulers produce schedules

- Since schedulers cannot execute operations before they are issued by the clients, schedulers delay operations.



transaction TA2 is waiting (aka blocked), $w_2[x]$ not yet executed

- The scheduling happens by waiting
- Local transactions are merged into a single *schedule*

Schedulers produce schedules

- Since schedulers cannot execute operations before they are issued by the clients, schedulers delay operations.

s: $r_1[x], r_1[y], r_3[z], w_1[x],$ $c_1, w_2[x], c_2, r_3[x], c_3$

TA1: $r_1[x], r_1[y],$ $w_1[x],$ c_1

TA2: $w_2[x]$ _____, c_2

TA3: $r_3[z],$ $r_3[x]$ _____, $c_3,$

- The scheduling happens by waiting
- Local transactions are merged into a single *schedule*

simple scheduler and disjoint-access TAs

- The simple scheduler does not delay disjoint-access TAs

Theorem: For the simple scheduler:

- If a transaction T is disjoint-access with all other transactions open during its lifetime (from Begin of Transaction of T to its commit/abort)
- then the simple scheduler does not delay this transaction.

Proof:

- The transaction does not encounter any lock by another transaction.

Result: The simple scheduler is **much** better than DB-wide mutual exclusion.

Example Deadlock for the simple scheduler

- Consider
- s: $r_1[x]$, $w_1[x]$, $r_2[y]$, $w_2[y]$, ????
- TA1: $r_1[x]$, $w_1[x]$, $r_1[y]$, _____???
- TA2: $r_2[y]$, $w_2[y]$, $r_2[x]$ _____???

Cyclical waiting cannot be resolved: deadlock

- A deadlock is a cyclical wait-relation between transactions in a scheduler.
- Deadlocks are detected by the scheduler (several approaches, e.g. graph-based detecting cycles in a waits-for graph).

Abort in non- DB-wide mutual exclusion

Problem: Implicit instant-online transaction management that do not block data disjoint transactions must sometimes abort transactions.

- Example: The request sequence:  $w_1[x], w_2[y], w_1[y], w_2[x]$ requires abort of one transaction.
- The prefix request schedule $w_1[x], w_2[y]$ cannot be blocked because it may be develop into data disjoint transactions. After this prefix has been executed, both transactions have done operations that are incompatible with any guaranteed correct completion.

Deadlock resolution

- If transactions wait in a cycle on each other (deadlock), abort one transaction in the cycle.
- We make use of the rollback mechanism that we already have defined.
- The transaction under rollback **keeps the locks** until the rollback is completed.
- The transaction to pick for abort may have to be chosen judiciously: we need to prevent starvation: that one transaction script always gets its transactions aborted.
- Deadlocks can be reduced by following patterns, e.g. naming a preferred order to access objects: if everyone follows the order, then no deadlocks appear!

Summary

1. **Transaction:** A sequence of operations that form a logical unit; the client has to request commit.
2. The simple scheduler (exclusive locks) does not delay disjoint-access transactions,
3. Works with one exclusive lock per object.
4. Locks are released at end of transaction
5. No explicit lock commands are needed.
6. Deadlocks can happen, require abort of a transaction.

Retry loop for transaction script

- In best practice for transactional programming, the transaction script must always be embedded in a *retry loop*:

```
do{  
    result = transactionScript(requestForm);  
} while(result.isRolledBack());
```
- All info for correct request answering must be collected beforehand as parameter for the request:
- Script has to be careful with communication outside DB.
- The loop can be implicitly given e.g. by certain containers for scripts, or messaging systems that do the retry, but we have to make sure that it happens somewhere.